

El bundle entra por stdin como un tar

Cómo llegan el código del alumno y los tests del profesor adentro de un contenedor sin red, con el rootfs de solo lectura y sin un solo montaje.

TPI · UTN FRC · Grupo 8 · TEMA 06 Base: 03 §1.5 · 08 §4-§8

1 El problema que resuelve

Un contenedor arranca desde una imagen. La imagen tiene el JDK, el jar de JUnit, el `entrypoint.sh` — todo lo que es igual en las 5.000 ejecuciones. Lo que **cambia en cada ejecución** es el código del alumno más los tests del profesor: unos pocos `.java`, algunas decenas de KB.

Esos archivos no están en la imagen. Hay que meterlos adentro de un contenedor que ya arrancó —o que está por arrancar— y que además está configurado para ser lo más hostil posible a que le escriban: `ReadonlyRootfs: true`, sin red, sin capacidades, con `/work` como un tmpfs.

Las tres formas de meter bytes son las de la tabla de 03 §1.5. Las dos primeras están descartadas; queda la tercera.

	Bind mount -v host:/work	docker cp	stdin (tar)
¿Expone un path del host?	Sí	No	No
¿Convive con <code>--read-only</code> ?	Sí	No	Sí
¿Convive con <code>--tmpfs</code> en el destino?	n/a	No	Sí
Qué necesita del sidecar	montajes	PUT <code>/archive</code> , que es una escritura	nada extra

El bind mount monta una carpeta real del host adentro del contenedor: no es una copia, son los mismos inodos. Es un agujero deliberado en el aislamiento del filesystem — si el alumno escapa de la JVM, ya tiene un pie en la máquina.

docker cp se descartó por dos fallas medidas en el prototipo: con el rootfs read-only el daemon rechaza la copia (`container rootfs is marked read-only`), y aunque fuera escribible, el `tmpfs` que se monta sobre `/work` al arrancar taparía lo copiado.

2 Qué es «el stdin de un contenedor»

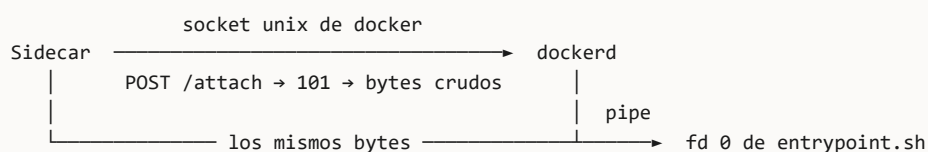
Todo proceso Unix nace con tres descriptores abiertos: `0` (stdin), `1` (stdout), `2` (stderr). El proceso PID 1 del contenedor —acá, `entrypoint.sh`— no es la excepción.

Cuando creás el contenedor con estos tres campos:

```
"OpenStdin": true,    // creá el fd 0 y dejalo abierto
"StdinOnce": true,    // cerralo cuando el que se adjuntó se desconecte
"AttachStdin": true   // voy a conectarme yo
```

Docker le arma al proceso un **pipe** en el fd 0, y el otro extremo queda del lado del daemon, esperando que alguien se conecte. Ese «alguien» es el sidecar.

La conexión se hace con `POST /containers/<id>/attach?stream=1&stdin=1`. Es una request HTTP sobre el socket de Docker, pero con `Upgrade: tcp` + `Connection: Upgrade`: el daemon responde **101 Switching Protocols** y a partir de ahí el socket deja de hablar HTTP. Es un caño de bytes crudo, exactamente como un WebSocket. Lo que escribas de este lado sale por el fd 0 del proceso de adentro.



No hay ningún path del host involucrado. No hay filesystem compartido, no hay volumen, no hay montaje. Es un caño de bytes — y un caño de bytes no se puede recorrer en la dirección contraria hacia el host. Eso es exactamente lo que buscábamos.

3 Por qué un tar y no otra cosa

Un pipe transporta **una secuencia de bytes**, no archivos. No tiene noción de «acá termina un archivo y empieza otro», ni de nombres, ni de directorios. Necesitás un formato que serialice un árbol de archivos en un stream lineal.

Eso es literalmente para lo que se inventó el tar: *tape archive*, un formato de 1979 para escribir directorios a una cinta magnética, que es el dispositivo secuencial por excelencia. Su estructura es la más tonta posible:

header 512 B	contenido	header 512 B	contenido	...
src/Sol.java	(pad a 512)	test/T.java	(pad a 512)	

Cada header trae nombre, tamaño, modo, uid/gid y tipo de entrada. Se lee de adelante hacia atrás, sin seeks y sin índice al final: podés extraerlo **a medida que llega**, sin conocer el tamaño total. Un zip no serviría igual de bien, porque su índice central está **al final** del archivo — necesitás el stream completo y poder hacer seek antes de extraer nada.

Por eso el `Content-Type` del `POST /ejecutar` es `application/octet-stream` con el tar **sin comprimir**. El bundle está topeado en 2 MiB de texto Java; comprimir agregaría un paso de descompresión adentro del contenedor —superficie de ataque, bomba de descompresión— a cambio de ahorrar unos KB en un socket local.

4 La secuencia exacta, paso a paso

#	Qué pasa	Por qué en ese orden
1	<code>POST /containers/create</code> con la spec de §4.1	El contenedor existe pero no corre
2	<code>POST /attach?stream=1&stdin=1</code> → espera 101	Antes del start (R5.1)
3	<code>POST /containers/<id>/start</code>	Ahora sí arranca <code>entrypoint.sh</code>
4	Escribir <code><nonce>\n</code> + <code><tar></code> y cerrar el socket entero	El cierre es lo que produce el EOF
5	<code>POST /wait?condition=not-running</code>	Bloquea hasta que el contenedor muere

El paso 2 va antes del 3, y no es cosmético. Si arrancás el contenedor primero, `entrypoint.sh` ejecuta su `cat >"$BUNDLE_TAR"` sobre un fd o al que todavía no se conectó nadie. En el mejor caso se queda esperando; en el peor lee EOF inmediato y muere con «bundle vacío en stdin».

El paso 4 es de una sola dirección (R5.2): se escribe y se cierra, nunca se lee de ese socket. Esa es la decisión que hace que el componente sea chico — no hay que multiplexar lectura y escritura sobre la misma conexión, no hay estado, no hay deadlocks de protocolo. La salida no vuelve por acá: vuelve después por `GET /logs`, que es una request HTTP normal.

5 El cierre — la parte que más se rompe

La primera línea real del `entrypoint` es:

```
cat >"$BUNDLE_TAR"
```

`cat` sin argumentos lee de `stdin` **hasta EOF**. Si el EOF nunca llega, `cat` se queda ahí para siempre y el contenedor nunca compila nada: termina en `TIMEOUT` a los 60 segundos y nadie entiende por qué.

El EOF del lado del lector se produce cuando el escritor **cierra su punta del pipe**. En la implementación Java eso es el `canal.shutdownOutput()` de `escribirEntrada` — un *half-close*: cierro mi dirección de escritura, el otro lado ve EOF. Más el `StdinOnce: true` de la spec del contenedor, que le dice al daemon que cuando el cliente adjunto se desconecte, cierre el fd o de adentro en vez de dejarlo abierto esperando otro `attach`.

Tres formas de arruinarlo, las tres reales

- **No cerrar.** `cat` cuelga y todo termina en timeout a los 60 s.
- **Cerrar el socket entero de golpe** en vez de hacer half-close, antes de que el kernel haya drenado el buffer de escritura: se pierden los últimos bytes, el tar llega truncado, `tar -tf` falla y el `entrypoint` muere con `BUNDLE_INVALIDO`.
- **Que el contenedor no consuma stdin.** El buffer de un pipe son unas decenas de KiB (64 KiB en Linux). Si el proceso de adentro no lee, tu `write` se bloquea al llenarlo.

R8.5, y es el caso más traicionero. Cuando la escritura se bloquea, el reloj de `TIMEOUT_EJECUCION_MS` **todavía no está corriendo** — ese reloj se evalúa recién en el paso 5. Sin un tope propio en la escritura, el hilo queda colgado para siempre reteniendo su cupo de concurrencia, y con `MAX_CONCURRENTES` casos así el ejecutor deja de atender **sin que /salud se entere**, porque saturación no es enfermedad (R3.3). Por eso escribir `Entrada` arma su propio watchdog que cierra el canal al vencer.

Y por eso R8.4 exige loguear, en cada ejecución, **cuántos bytes se escribieron en stdin y si el cierre se completó**: desde afuera, un `TIMEOUT` es indistinguible entre un `while(true)` del alumno y un half-close mal hecho tuyo. Sin ese log no hay forma de saber a quién culpar.

6 El nonce: el pasajero que viaja adelante del tar

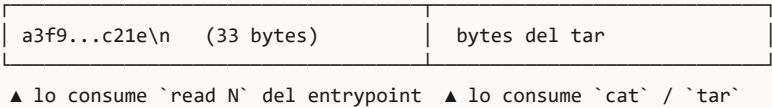
Acá está la parte más ingeniosa del mecanismo, y es §7 de la spec.

El reporte de JUnit tiene que volver por `stdout`. Pero por ese mismo `stdout` escribe el código del alumno. Si el separador entre «salida del alumno» y «reporte» fuera una constante conocida, un `System.out.println("---SANDBOX-INICIO---...")` **falsifica el veredicto**. No es una fuga de aislamiento: es una falsificación por un canal legítimo, y ninguna flag de Docker la cubre.

La solución es un nonce aleatorio de 16 bytes (32 hex) por ejecución, que el alumno no puede adivinar. Pero, ¿cómo se lo hacés llegar al entryptpoint sin que el alumno lo lea?

Canal	Por qué no sirve
Variable de entorno	El alumno lee <code>/proc/1/envIRON</code> . Y <code>unset</code> no lo borra de ahí : cambia la memoria del shell, no el snapshot que el kernel tomó en el <code>execve</code>
Argumento del comando	<code>/proc/1/cmdline</code>
Archivo en el tmpfs	El alumno tiene el mismo uid y el mismo <code>/tmp</code>

La solución es el stdin, aprovechando que ya lo estamos usando (R7.2): el nonce viaja como **primera línea**, terminada en `\n`, y el tar viene inmediatamente después, pegado, en el mismo stream.



El entryptpoint hace `read N` en una variable **no exportada** — así no aparece en el `envIRON` de ningún hijo— y después `tar` sigue leyendo desde donde `read` dejó. El nonce nunca toca el disco ni el entorno: vive en la memoria del shell PID 1, que es el único proceso que el alumno no controla.

El supuesto que había que verificar

Todo esto depende de que `read` **no lea de más**. Un `read` eficiente leería un bloque de 4 KB y se quedaría con el sobrante — y ahí te comiste el principio del tar. Los shells POSIX leen **de a un byte** sobre descriptores no posicionables justamente para no romper esto, pero la spec no lo daba por sentado: hay un fixture dedicado (`src/test/fixtures/a6/entrypoint.sh`) que existe solo para probarlo.

```
read N
echo "FIN $N"
echo "---SANDBOX-$N-INICIO---"
tar -tf -          # ← si `read` leyó de más, esto falla
echo "---SANDBOX-$N-FIN---"
```

Del lado de la vuelta, R7.5 fija el criterio de extracción: **la última aparición del marcador de inicio, y la primera de fin posterior a ella**. No porque el alumno pueda producir el nonce —no puede—, sino para que dos implementaciones (Java y Node) nunca diverjan ante el mismo stream.

7 El tar es entrada hostil

El tar entra por un canal seguro, pero **su contenido lo controla el alumno**. Esta es la familia de bugs que produjo los dos CVE 10.0 de Judgeo y los tres de `runc` de noviembre de 2025. Por eso el entryptpoint valida **antes de extraer**, no después:

```
cat >"$BUNDLE_TAR"          # 1. al disco (tmpfs), sin extraer
tar -tf "$BUNDLE_TAR" >rutas # 2. LISTAR, que no escribe nada

while read ruta; do          # 3. validar cada ruta
  case "$ruta" in
    /*)          morir "ruta absoluta" ;;
    *.* )        morir "ruta con .." ;;
    src/*|test/*) : ;;
    *)           morir "ruta fuera de src/ y test/" ;;
  esac
done <rutas

tar -xf "$BUNDLE_TAR" -C "$DIR_IN" \    # 4. recién ahora
    --no-same-owner --no-same-permissions --no-overwrite-dir
```

`--no-same-owner` es lo que impide que el tar dicte el uid de los archivos extraídos — el vector exacto de CVE-2024-28189, donde el investigador `bypasseó` el parche de Judgeo haciendo `chown` sobre un symlink.

Y esto es la **segunda** barrera: la API ya validó las rutas en el JSON de entrada (relativa, sin `..`, sin barra inicial, terminada en `.java`). El entryptpoint valida de nuevo porque es el único que ve el tar real. Defensa en profundidad — si la API tiene un bug, acá no pasa nada.

Por encima de las dos, la red de contención estructural: aunque un traversal se colara, `ReadonlyRootfs: true` hace que el `write` fuera de `/tmp` falle en el kernel. Por eso la tabla de §4.2 dice que read-only es **innegociable: es lo que hace que un path traversal exitoso igual no sirva de nada**.

Lo que falta probar. El vector *symlink* y el *hard link*. El `case` de arriba filtra `..` y rutas absolutas, pero un tar que contiene un symlink `src/X.java → /etc/passwd` y después una entrada regular que escribe sobre ese path **no tiene ningún `..` en ninguna ruta**. Son dos casos más sobre los diez que ya existen en la suite de entregas maliciosas.

8 Beneficio colateral para el sidecar

Si nadie usa `docker cp`, la lista blanca del proxy **no necesita habilitar** `PUT /containers/{id}/archive` — que es, literalmente, una escritura arbitraria al filesystem de un contenedor.

La superficie del daemon que el ejecutor necesita queda en: `create`, `attach`, `start`, `wait`, `json`, `logs`, `delete`, `kill`, `_ping`. **Ninguna escribe archivos.**

Pendiente detectado: la spec y el prototipo divergen

La spec §7 y la implementación Java (`Ejecucion.java:153`, `Reporte.java:18`) usan el nonce. Pero `sandbox/runner/entrypoint.sh` —el entrypoint de la imagen— todavía usa marcadores fijos:

```
readonly MARCA_INI='##SANDBOX-RUNNER-V1##'
readonly MARCA_FIN='##FIN-SANDBOX-RUNNER##'
```

y arranca con `cat >"$BUNDLE_TAR"` directo, **sin el read N previo**. Contra esa imagen, el ejecutor mandaría el nonce como primera línea y el `cat` se lo tragaría adentro del tar —corrompiéndolo—, y además los marcadores que el ejecutor busca nunca aparecerían.

Son dos implementaciones de dos épocas distintas. No se dispara hoy, porque el ejecutor Java corre contra un daemon de prueba y no contra esa imagen, pero **hay que cerrarlo antes de conectarlos**.

Dónde vive cada cosa

<code>docs/arquitectura/03-ms-sandbox-ejecucion.md §1.5</code>	La comparación de las tres vías y por qué gana stdin
<code>docs/arquitectura/08-spec-ejecutor.md §4</code>	La spec completa del contenedor, campo por campo
<code>docs/arquitectura/08-spec-ejecutor.md §5</code>	La secuencia de 7 llamadas al daemon (R5.1–R5.10)
<code>docs/arquitectura/08-spec-ejecutor.md §7</code>	Nonce, entrada y separación del reporte (R7.1–R7.8)
<code>docs/arquitectura/08-spec-ejecutor.md §8</code>	Timeouts, incluido el tope propio de la escritura (R8.5)
<code>sandbox/runner/entrypoint.sh</code>	El lado receptor: validación, extracción y las tres fases
<code>java sidecar</code>	El lado emisor: attach, escritura y half-close
<code>test/src/main/java/sandbox/ejecutor/Ejecucion.java</code>	